

TaskFlow

Module : Conception Logicielle

Présenté par :

-Salwa Hafid.

-Sara El ghazine.

- Soumaya Ait elkabir.

- Souad Iaraben.

Encadré par : Pr. LABBIHI

Année Universitaire :

2025-2026.

Filière : CDL

Introduction

Dans le cadre du module de **Conception Logicielle**, ce projet consiste à concevoir une application de gestion de tâches collaborative nommée **"TaskFlow"**. L'objectif principal est de mettre en pratique les principes fondamentaux de l'architecture logicielle, notamment la modélisation **UML** et l'application des **Design Patterns**, tout en respectant les principes de qualité tels que **SOLID** et **KISS**.

Analyse des Besoins

• Besoins Fonctionnels :

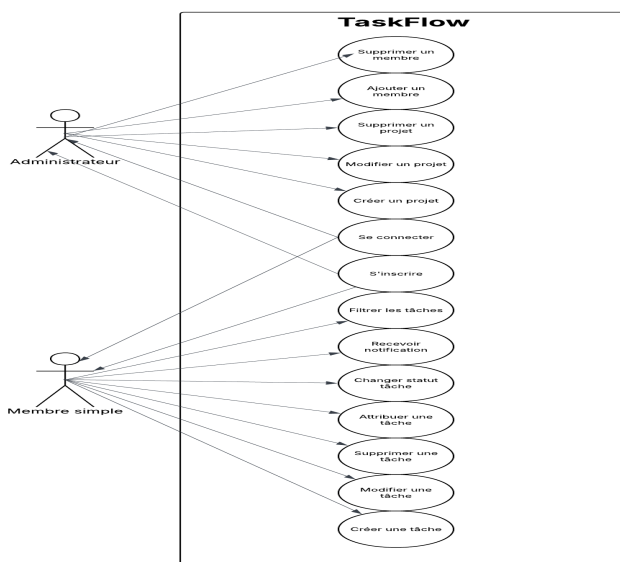
- **Gestion des accès** : Inscription et connexion sécurisée (Admin/Membre).
- **Gestion des projets** : Création de projets et gestion des membres.
- **Suivi des tâches** : Création, modification et changement de statut (À faire, En cours, Terminée).
- **Collaboration** : Attribution des tâches et notifications automatiques.

• Besoins Non-Fonctionnels :

- **Sécurité** : Protection des données.
- **Maintenabilité** : Utilisation des principes **SOLID** et **KISS** pour un code propre.

Les Diagrammes

• Diagramme de cas d'utilisation :



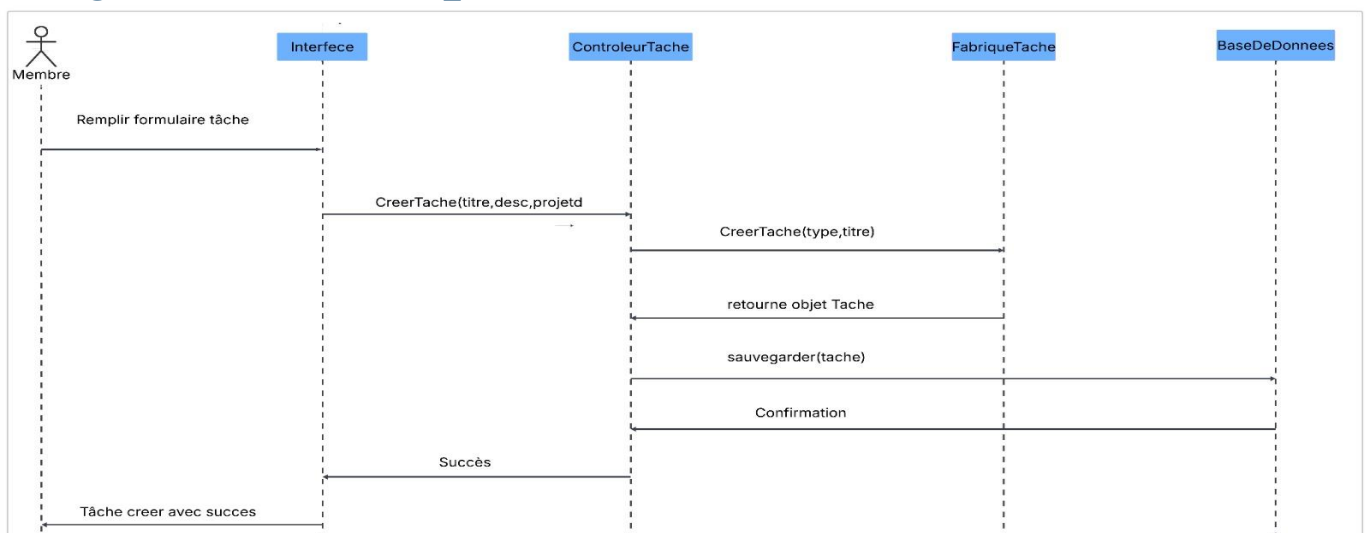
- Ce diagramme de cas d'utilisation illustre les interactions fonctionnelles au sein du système **TaskFlow** entre deux acteurs principaux. L'**Administrateur** assure la gestion structurelle en créant et modifiant les projets ainsi qu'en gérant l'adhésion des membres. Parallèlement, le **Membre simple** se concentre sur l'aspect opérationnel : il crée, attribue et met à jour le statut des tâches (À faire, En cours, Terminée) tout en recevant des notifications automatiques lors de ses assignments. L'ensemble de ces fonctionnalités, incluant l'authentification et le filtrage des tâches, garantit un flux de travail collaboratif organisé et sécurisé.

• Diagramme de classes :



- Ce diagramme de classes illustre la structure statique du système **TaskFlow**. Il se compose de quatre entités principales : **Utilisateur**, qui gère les profils et l'authentification ; **Projet**, qui sert de conteneur pour regrouper les membres et les travaux ; **Tache**, l'élément central définissant les activités à réaliser avec leurs états respectifs ; et **Notification**, responsable de l'alerte des utilisateurs lors d'événements clés. Les relations montrent qu'un utilisateur peut piloter plusieurs projets et tâches, tandis qu'une tâche est rattachée à un projet unique et peut générer plusieurs notifications pour assurer un suivi collaboratif efficace.

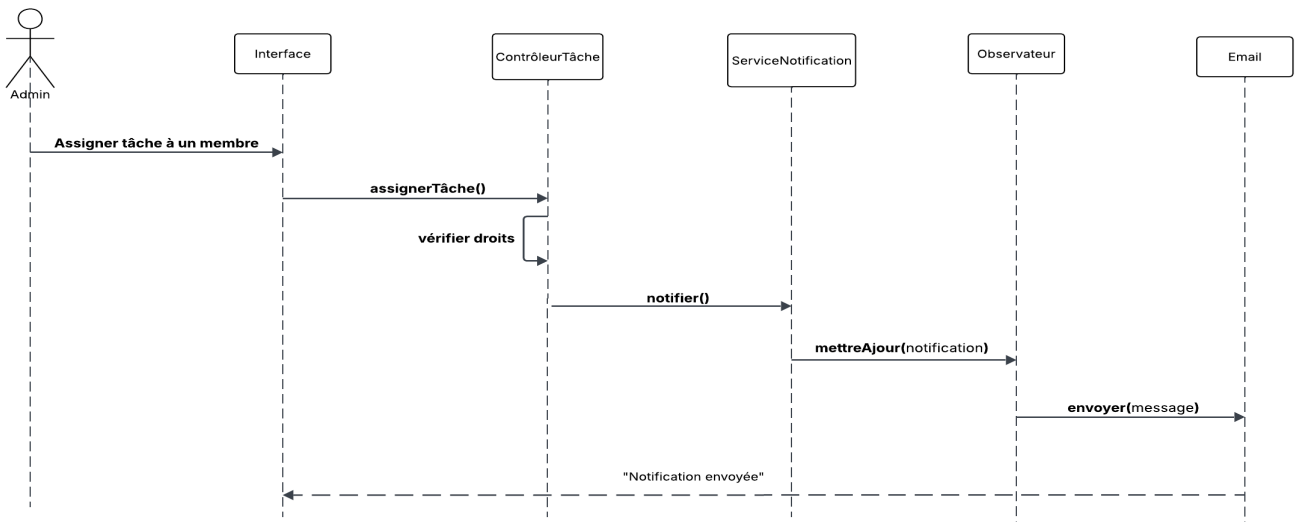
• Diagramme de séquence 1 :



- Ce diagramme de séquence détaille le processus de création d'une nouvelle tâche par un **Membre**. Le flux débute par la saisie du formulaire via l'**Interface**, qui transmet les données au **ContrôleurTache**. Ce dernier sollicite la **FabriqueTache** (Pattern Factory) pour instancier l'objet, puis assure sa persistance dans la **BaseDeDonnees**. Le processus se clôture par un message de

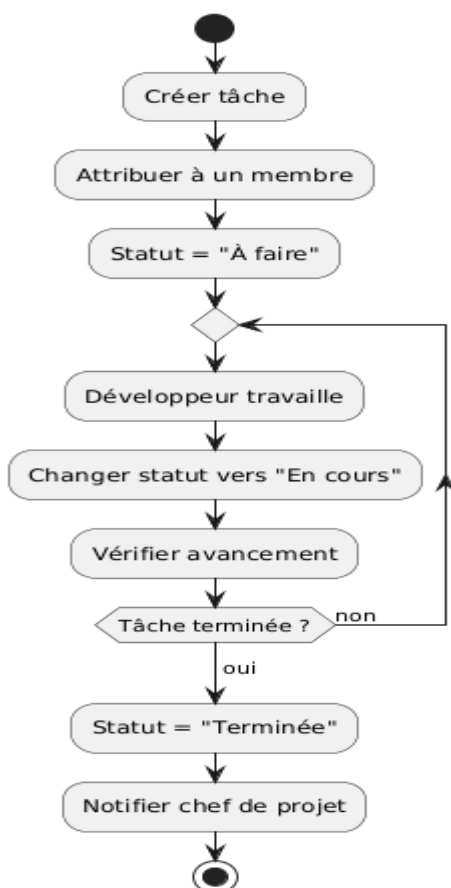
confirmation retourné à l'utilisateur, garantissant ainsi une traçabilité complète de l'action au sein de l'architecture.

• Diagramme de séquence 2 :



- Ce diagramme de séquence illustre le processus d'assignation d'une tâche et le système de notification associé, impliquant l'acteur **Admin** et plusieurs composants techniques. Le flux commence par l'assignation via l'**Interface**, suivie d'une vérification des droits par le **ContrôleurTâche**. Une fois validée, la demande déclenche le **ServiceNotification** qui utilise un **Observateur** pour mettre à jour et envoyer un message final par **Email**, confirmant ainsi la bonne réception de la notification à l'administrateur.

• Diagramme d'activité :



- Ce diagramme d'activité modélise le **cycle de vie opérationnel d'une tâche** au sein de l'application. Le processus débute par la création et l'attribution de la tâche à un membre, l'initialisant à l'état "**À faire**". Il décrit ensuite un flux itératif où le développeur travaille et passe le statut à "**En cours**" jusqu'à ce que la vérification de l'avancement confirme que la tâche est terminée. Une fois validée ("**Oui**"), le statut final est mis à jour à "**Terminée**" et une notification automatique est envoyée au chef de projet pour clôturer l'activité.

Design Patterns utilisés

Afin d'améliorer l'organisation et la maintenabilité de l'application TaskFlow, plusieurs design patterns ont été utilisés dans la conception du système.

- **Singleton :**

Le pattern Singleton a été utilisé pour la gestion des notifications afin de garantir une seule instance du service de notification dans toute l'application.

- **Observer :**

Le pattern Observer a été utilisé pour notifier automatiquement les membres lorsqu'une tâche leur est assignée ou modifiée.

- **Factory :**

Le pattern Factory permet de créer différents types d'utilisateurs (administrateur ou membre) de manière flexible et centralisée

Environnement Technique

- **Modélisation :** UML.
- **Langage :** Java.
- **Versiionnage :** Git & GitHub pour le travail d'équipe.
- **Outils :** StarUML et Lucidchart pour les diagrammes.
- **Documentation :** Javadoc pour le code et Swagger pour l'API.

Conclusion

Ce mini-projet nous a permis d'appliquer les concepts de conception logicielle à travers la modélisation UML, l'utilisation des design patterns et l'organisation d'une architecture claire et maintenable.

Le projet TaskFlow nous a également aidés à comprendre l'importance de la documentation technique et du travail collaboratif avec Git et GitHub.

Cette expérience a renforcé nos compétences en analyse, conception et structuration d'applications logicielles.